AntiVAX ISA Specification

Tristan Miller, Rose Novack, Robert White

March 2026

1 Introduction

AntiVAX is a micro-coded CISC instruction set featuring a variety of addressing modes and vector instructions. It seeks to infinitely upon VAX's inferior design.

2 Instruction Encoding

Instructions in AntiVAX are variable length encoded. Each opcode expects a certain number of operands (no more and no less). With a few exceptions, all operands in an instruction can be any addressing mode (i.e. memory, register, immediate). All comparisons, unless otherwise specified, assume signed numbers.

Instruction Format:

2 bit pad	6 bit opc	operands

Operand restrictions:

- destination cannot be an immediate (you cannot write to a number)
- array pointers (i.e. `arr1`, `arr2`) MUST be in memory (i.e. not registers or immediate)

Each operand begins with a mode byte. The high 3 bits specify the addressing mode of the operand while the low 5 specify a register. Some addressing modes require additional bytes for another register or an immediate – see the addressing mode table for more details.

3 bit am	5 bit reg

3 Registers

The address of the next instruction is stored in a 32-bit program counter. This register cannot be altered directly but can be read and modified through control flow instructions.

There are 32 program-accessible registers. Their recommended uses are listed below.

Index (Hexadecimal)	Asm. Name	Description
`00`	`pc`	Pointer to the instruction to be executed next
`01`	`ra`	Return address for function calls
`02`	`fp`	Address of the start of the current stack frame
`03`	`sp`	Address of the top element in the stack
`04-07`	`a0-a3`	Function arguments. `a0` and `a1` are additionally used to return values from a function
`08-0f`	`s0-s7`	Callee-saved registers
`10-1f`	`t0-t15`	Caller-saved (temporary) registers

4 Memory

The memory is byte-addressable with a 32-bit data bus. Reads and writes of word-size (32-bit) data must be aligned, but instructions need not be aligned. Data is stored in little-endian order.

The stack grows towards higher addresses. The stack pointer points to the address immediately higher than the top of the stack. It is the user's responsibility to set the initial stack pointer location.

Matrices are stored in row-major order.

5 Addressing modes

In this table, `#` represents an immediate value, `R` and `S` represent arbitrary registers, and `d` is the data size of the instruction.

Name	Enc.	Asm. Syn	EA	Description
Register	0	`R`	—	Read/write to a register
Immediate	1	`#`	—	Immediate value
Absolute	2	`[#]`	`mem[#]`	Address given immediate
Displacement	3	`[R + #]`	`mem[R+#]`	Register offset by immediate
Indexed	4	`[R + d*S]`	`mem[R+d*S]`	Register offset by scaled register
Memory indirect	5	`[[R]+#]`	`mem[mem[R]+#]`	Pointer offset by immediate
Postincrement	6	`[R+]`	`mem[R]; R += d`	Address given by register, register incremented
Predecrement	7	`[-R]`	`R -= d; mem[R]`	Register decremented, address given by register

The immediate, absolute, displacement, and memory indirect addressing modes require a 4-byte immediate after the mode byte. The indexed addressing mode requires a 1-byte secondary register.

6 Instructions

Unused opcodes are left blank.

	0	1	2	3	4	5	6	7
00	halt	nop	fnop		move		pushs	pops
08	jump	jal			beq	bne	blt	ble
10	add	sub	mul	div	sdiv	and	or	xor
18	eq	ne	lt	lte	sll	srl	sra	not
20	addarr	subarr	mularr		saddarr	ssubarr	smularr	
28	copyarr	setarr			dotarr	mulmat	plyeval	
30	dmpval	dmpall	dmpmemrng					
38								

See "Addressing Modes" for how operands are formatted.

Name	Opc	Description	Operands (in order)
halt	00	halts the cpu	
nop	01	no-op	
fnop	02	no-op but more fun	
move	04	moves op1 to dst	dst, op1
pushs	06	push all “S” registers	
pops	07	pop all “S” registers	
jump	08	pc = op1	op1
jal	09	jump and link (sets ra): pc = op1, ra = pc	op1
beq	0C	branch if equal: if op1 == op2, pc = op3	op1, op2, op3
bne	0D	branch if not equal: if op1 != op2, pc = op3	op1, op2, op3
blt	0E	branch if less than: if op1 < op2, pc = op3	op1, op2, op3
ble	0F	branch if less than or equal: if op1 <= op2, pc = op3	op1, op2, op3
add	10	dst = op1 + op2	dst, op1, op2
sub	11	dst = op1 - op2	dst, op1, op2
mul	12	dst = op1 * op2	dst, op1, op2
div	13	unsigned division: dst = op1 / op2	dst, op1, op2
sdiv	14	signed division: dst = op1 / op2	dst, op1, op2
and	15	dst = op1 & op2 (bitwise)	dst, op1, op2
or	16	dst = op1 | op2 (bitwise)	dst, op1, op2
xor	17	dst = op1 ^ op2 (bitwise)	dst, op1, op2
eq	18	dst = op1 == op2	dst, op1, op2
ne	19	dst = op != op2	dst, op1, op2
lt	1A	dst = op1 < op2	dst, op1, op2
lte	1B	dst = op1 <= op2	dst, op1, op2
sll	1C	shift left logical: dst = op1 << op2	dst, op1, op2
srl	1D	shift right logical: dst = op1 >> op2	dst, op1, op2
sra	1E	shift right arithmetic: dst = op1 >>a op2	dst, op1, op2
not	1F	dst = ~op1 (bitwise)	dst, op1

Name	Opc	Description	Pseudocode	Operands (in order)
addarr	20	adds two arrays (or matrices)	`for i in len:` `  arrdst[i] = arr1[i] + arr2[i]`	arrdst, arr1, arr2, len
subarr	21	subtracts two arrays (or matrices)	`for i in len:` `  arrdst[i] = arr1[i] - arr2[i]`	arrdst, arr1, arr2, len
mularr	22	multiplies each element in array 1 by each element in array 2	`for i in len:` `  arrdst[i] = arr1[i] * arr2[i])`	arrdst, arr1, arr2, len
saddarr	24	add a scalar to each element of a arrays	`for i in len:` `  arrdst[i] = arr1[i] + op1`	arrdst, arr1, len, op1
ssubarr	25	subtract a scalar from each element of an array	`for i in len:` `  arrdst[i] = arr1[i] - op1`	arrdst, arr1, len, op1
smularr	26	multiplies each element in an array by a scalar	`for i in len:` `  arrdst[i] = arr1[i] * op1`	arrdst, arr1, len, op1
copyarr	28	copies an array to some other location	`for i in len:` `  arrdst = arr1[i]`	arrdst, arr1, len
setarr	29	set an array to a scalar	`for i in len:` `  arrdst = op1`	arrdst, len, op1

dotvec	2C	takes the dot product of two vectors	`dst = arr1[0] * arr2[0]`
			`  + ...`
			`  + arr1[len-1] arr2[len-1]`
				dst, arr1, arr2, len

mulmat	2D	Multiplies together two matrices. The matrix dimensions are m x n and n x o.	`arrdst = arr1 x arr2`	arrdst, arr1, arr2, m, n, o
plyeval	2E	given an array of constants representing coefficients in a polynomial form arr[0] + arr[1]x + arr[2]x^2 +... +arr[n]x^n and a value x, calculates the value of the equivalent polynomial		dst, arr, len, x
dmpval	30	prints op1's value (debug)		op1
dmpall	31	dumps all registers' values (debug)		
dmpmemrng	32	dumps a memory range (debug)		arr, len